Two-Machine Flow Shop Scheduling to Minimize Total Weighted Completion Time: Structural Properties and Dynamic Programming

Author One[a,*], Author Two[b]

[a]Institution One, Location, Country

[b]Institution Two, Location, Country

Abstract

Minimizing the total weighted completion time on a two-machine flow shop, denoted $F2 \parallel \sum w_j C_j$, is a classical NP-hard scheduling problem with applications in manufacturing, computing, and logistics. Existing exact methods rely on enumerative techniques that scale poorly with instance size, while approximation algorithms often impose restrictive assumptions on processing times. We first sequence all jobs globally by Johnson's rule (SPT(1)-LPT(2)), obtaining an initial permutation S. Scanning S from left to right, we partition it into at most K blocks: whenever a weight not seen before appears, it serves as the final job of the current block, and the next job starts a new block. We establish four structural properties of optimal schedules: the permutation property, the positional constraint that block B_m jobs cannot appear beyond the first m blocks, the block-end condition requiring the final job of each block to carry that block's weight (automatically satisfied by construction), and the intra-block Johnson ordering inherited from S. Building on these properties and the interval-based dynamic programming framework developed for $F2|r_j|C_{\max}$, we design a polynomial-time dynamic programming algorithm with $O(|Y|_{\max} \cdot n)$ complexity that appends jobs block by block. We further show that geometric rounding of weights and processing times converts this algorithm into a polynomial-time approximation scheme.

Keywords: Flow shop scheduling; Total weighted completion time; Dynamic programming; Structural properties

*Corresponding author. E-mail address: author@email.com

1 Introduction

2 Problem Formulation

3 Structural Properties

There are at most K distinct weights among the n jobs. Let K denote the number of distinct weight values that actually appear.

We first sort all n jobs (regardless of weight) by Johnson's rule (SPT(1)-LPT(2)), obtaining a global sequence

$$S = (j_1, j_2, \ldots, j_n).$$

We partition S into blocks $B_1, B_2, \ldots, B_M$ ($M \leq K$) as follows. Maintain a set of weights already seen and a pointer *start* marking the beginning of the current block, initially *start* $=$ 1. Scan S from $i = 2$ to n: whenever $w_{S[i]}$ has never been seen before, close the current block as $B_m = S[start \ldots i]$ (the new-weight job is the *last* job of the block), increment the block counter, and set $start = i + 1$ for the next block. After the scan, any remaining jobs form the final block. The total number of blocks M satisfies $M \leq K$, because each block is closed by the first occurrence of a distinct weight.

Each block B_m is a contiguous subsequence of S, and also serves as the ordered list of jobs belonging to that block. Let $W(B_m)$ denote the weight of the final job of B_m (the new weight that triggered the cut); by construction, $W(B_m)$ is the unique weight in B_m that had not appeared in any earlier block. Let $|B_m|$ denote the number of jobs in B_m, with $\sum_{m=1}^{M} |B_m| = n$.

Lemma 3.1 ***(Permutation property.)*** *For $F2 \parallel \sum w_j C_j$, there exists an optimal schedule in which the job sequence is identical on both machines.*

This is a classic result for two-machine flow shops with regular objective functions.

Lemma 3.2 ***(Positional constraint.)*** *In any optimal schedule, for each $m = 1, \ldots, M$, the jobs belonging to block B_m can only appear within the first m blocks of the final schedule.*

Lemma 3.2 implies that once the m-th block of the final schedule has been processed, all jobs from the first m blocks of S are fully scheduled. Consequently, the m-th final block may contain only jobs from $B_m, B_{m+1}, \ldots, B_M$.

Lemma 3.3 ***(Block-end condition.)*** *In any optimal schedule, the final job of the m-th final block carries weight $W(B_m)$, for each $m = 1, \ldots, M$. By the block construction rule, $W(B_m)$ is the weight that first appears at the end of B_m in S; thus the condition is automatically satisfied.*

Lemma 3.4 ***(Intra-block ordering.)*** *Within each final block, all jobs are sequenced according to the SPT(1)-LPT(2) rule (Johnson's rule): for any two jobs x, y belonging to the same final block, job x precedes job y if and only if*

$$\min\{a_x, b_y\} < \min\{a_y, b_x\},$$

with ties broken by non-decreasing a_j. Since the initial Johnson sequence S already satisfies this ordering globally, and each block B_m is a contiguous subsequence of S, the intra-block Johnson order is preserved from S.

Taken together, these four properties determine the structure of any optimal schedule: all jobs are first sequenced globally by Johnson's rule, then partitioned into $M \leq K$ blocks by the first occurrence of each new weight; the final schedule respects the block order (block B_m cannot appear beyond the m-th final position); the last job of each block carries the weight $W(B_m)$; and each block internally preserves the Johnson ordering inherited from S.

4 Dynamic Programming Algorithm

The structural properties established in Section 3 enable a dynamic programming algorithm that constructs the schedule by appending jobs block by block. Our approach adapts the interval-based state representation of **?**, originally devised for the $F2|r_j|C_{\max}$, to the weighted completion time objective.

The preprocessing consists of two steps. First, all n jobs (regardless of weight) are sorted by Johnson's rule (SPT(1)-LPT(2)) to obtain an initial sequence $S = (j_1, j_2, \ldots, j_n)$. Second, S is partitioned into $M \leq K$ blocks by scanning for the first occurrence of new weights: let *seen* be the set of weights encountered so far (initially $\{w_{S[1]}\}$) and *start* the beginning of the current block (initially 1). For $i = 2, \ldots, n$, if $w_{S[i]} \notin$ *seen*, the current block is closed as $B_m = S[start \ldots i]$ (the new-weight job is the last of the block), *start* is advanced to $i + 1$, and $w_{S[i]}$ is added to *seen*. After the scan, if *start* $\leq n$ the remaining jobs form the final block. Each B_m is a contiguous subsequence of S and serves as the ordered job list for that block.

The algorithm processes jobs in block order $B_1, B_2, \ldots, B_M$, appending each job to the end of the current partial permutation. The positional constraint of Lemma 3.2 is automatically satisfied by this discipline: jobs of B_m are appended only after blocks $B_1, \ldots, B_{m-1}$ have been fully processed. The block-end condition is enforced at block boundaries via the check $W(B_m) \leq \ell$ described below. Its correctness follows from the fact that $W(B_m)$ is the weight of the final job of block B_m and $\ell = W(B_{m-1})$; the inequality ensures block weights are non-increasing, a necessary condition for optimality. If violated, no subsequent appending from $B_{m+1}, \ldots, B_M$ can alter the already-fixed weight ordering of the prefix, so the state

can be safely discarded. If the check passes, ℓ is updated to $W(B_m)$ for comparison with the next block.

4.1 State Definition and Dynamic Programming

Each state implicitly corresponds to a partial permutation π uniquely determined by the DP construction path; π is not stored. A state is a 4-tuple $(A,\, \mathcal{B},\, z,\, \ell)$, where A and $\mathcal{B}$ are the cumulative completion times on M1 and M2, respectively, $z = \mathcal{B}$ is the makespan of the partial schedule, and ℓ is the reference weight $W(B_{m-1})$ of the previously completed block. The initial state is $(0,\, 0,\, 0,\, +\infty)$, corresponding to $\pi = \emptyset$, where $\ell = +\infty$ ensures that the first block always passes the weight check.

Let $\mathcal{Y}$ denote the set of states after processing a given prefix of jobs. Given a state $(A,\, \mathcal{B},\, z,\, \ell) \in \mathcal{Y}$ and a job $j \in B_m$ to append, the transition produces a new state $(A',\, \mathcal{B}',\, z',\, \ell')$ defined by

$$A' = A + a_j, \tag{1}$$

$$\mathcal{B}' = \max(\mathcal{B},\, A') + b_j, \tag{2}$$

$$z' = \mathcal{B}', \tag{3}$$

$$\ell' = \begin{cases} W(B_m), & \text{if } j \text{ is the last job of } B_m, \\ \ell, & \text{otherwise.} \end{cases} \tag{4}$$

All four operations take $O(1)$ time. At a block boundary (when the last job of B_m is appended), the constraint $W(B_m) \le \ell$ is verified; if violated, the candidate state is discarded.

Lemma 4.1 *For any two states $(A, \mathcal{B}, z, \ell)$ and $(A', \mathcal{B}', z', \ell')$ in $\mathcal{Y}$ satisfying $z \le z'$, the latter state can be removed from $\mathcal{Y}$.*

Proof. Let ρ and ρ' be the partial schedules attaining $(A, \mathcal{B}, z, \ell)$ and $(A', \mathcal{B}', z', \ell')$, respectively. Let σ be an arbitrary suffix schedule for the remaining jobs. Appending σ to ρ yields a complete schedule, and appending σ to ρ' yields another. Since the contribution of σ to the makespan is monotone non-decreasing in the prefix completion times $(A, \mathcal{B})$, the condition $z \le z'$ guarantees that ρ leads to a makespan no larger than ρ' under any suffix. Therefore ρ dominates ρ'. □

The dominance rule of Lemma 4.1 justifies retaining only states with the minimum z value after processing each job. The complete DP procedure is presented as Algorithm 1.

Algorithm 1 DP_F2_WeightedSum

Input: Jobs J with processing times (a_j, b_j) and weights w_j, $j = 1, \ldots, n$.

Step 1. (Preprocessing) Sort all jobs by Johnson's rule to obtain $S = (j_1, \ldots, j_n)$. Partition S into blocks $B_1, \ldots, B_M$ ($M \leq K$) by the first occurrence of new weights.

Step 2. (Initialization) Set $\mathcal{Y} \leftarrow \{(0,\, 0,\, 0,\, +\infty)\}$.

Step 3. (Generation) Process blocks $m = 1, \ldots, M$ sequentially.

For $m = 1$ to M **do**

$w_m \leftarrow W(B_m)$

For $i = 1$ to $|B_m|$ **do**

$j \leftarrow B_m[i]$; $\quad \mathcal{Y}_{\text{new}} \leftarrow \emptyset$

For each $(A,\, \mathcal{B},\, z,\, \ell) \in \mathcal{Y}$ **do**

$A' \leftarrow A + a_j$; $\quad \mathcal{B}' \leftarrow \max(\mathcal{B},\, A') + b_j$; $\quad z' \leftarrow \mathcal{B}'$

If $i = |B_m|$ **then**

If $w_m > \ell$ **then continue**

$\ell' \leftarrow w_m$

Else $\ell' \leftarrow \ell$

End if

$\mathcal{Y}_{\text{new}} \leftarrow \mathcal{Y}_{\text{new}} \cup \{(A',\, \mathcal{B}',\, z',\, \ell')\}$

End for

(**Elimination**) $z_{\min} \leftarrow \min\{\, z' \mid (\cdot,\, \cdot,\, z',\, \cdot) \in \mathcal{Y}_{\text{new}} \,\}$;
$\mathcal{Y} \leftarrow \{\, s \in \mathcal{Y}_{\text{new}} \mid s.z = z_{\min} \,\}$

End for

End for

Step 4. (Result) Set $Z^* \leftarrow \min\{\, z \mid (A,\, \mathcal{B},\, z,\, \ell) \in \mathcal{Y} \,\}$, and recover the optimal schedule by backtracking.

Output: The optimal makespan Z^* and its associated optimal permutation π^*.

4.2 Complexity Analysis

Let $|\mathcal{Y}|_{\max}$ denote the maximum number of states retained after the elimination step in any single iteration. In the append-only regime each parent state produces exactly one child state, so the state count never explodes combinatorially. The per-state transition takes $O(1)$

time (three arithmetic operations and one comparison). With n jobs processed sequentially, the total running time of Algorithm 1 is

$$O\big(|\mathcal{Y}|_{\max} \cdot n\big). \tag{5}$$

The elimination rule of Lemma 4.1 keeps $|\mathcal{Y}|_{\max}$ small in practice: for each distinct $(A, \mathcal{B}, \ell)$ combination, at most one state (with minimum z) is retained. The $W(B_m) \leq \ell$ checks at block boundaries further prune infeasible states early. Each state stores four scalar fields, requiring $O(1)$ space per state, and the total space complexity is $O(|\mathcal{Y}|_{\max})$.

The number of blocks satisfies $M \leq K$. Under geometric rounding of weights, $K = O((1/\varepsilon) \log W_{\max})$, so M is bounded by a polynomial in $1/\varepsilon$ independent of n. Moreover, the elimination step can be implemented in $O(|\mathcal{Y}_{\text{new}}|)$ time by a single linear scan that tracks the minimum z'.

Theorem 4.1 *Algorithm 1 finds an optimal schedule for $F2 \parallel \sum w_j C_j$ in $O(|\mathcal{Y}|_{\max} \cdot n)$ time and $O(|\mathcal{Y}|_{\max})$ space.*

Proof. Correctness follows from Lemmas 3.1–3.4, the block partitioning rule of Section 3, the state transition (1)–(4), and the dominance rule of Lemma 4.1. Step 1 (Johnson sorting and block partitioning) takes $O(n \log n)$ time. Steps 2 and 4 require $O(1)$ and $O(|\mathcal{Y}|)$ time, respectively. In Step 3, the outer loop over m and inner loop over i together visit each of the n jobs exactly once. For each job, the algorithm iterates over $|\mathcal{Y}|$ states and performs $O(1)$ work per state, then applies elimination in $O(|\mathcal{Y}_{\text{new}}|)$ time. Hence the total cost is $O(|\mathcal{Y}|_{\max} \cdot n)$. The space bound follows because only $\mathcal{Y}$ and $\mathcal{Y}_{\text{new}}$ are stored, each of size at most $|\mathcal{Y}|_{\max}$. □

5 Conclusion